



Semester Project

Re-engineering a Legacy System

I222614_Anishra Ashfaq

I222678_Ayesha Khalid

I222718_Aisha Adil

CS 4001 - Software Reengineering

Instructor: Madam Anum Kaleem

Monday, 1 December 2025.

Contents

Introduction	5
Objective	5
Scope	5
Project Repository	5
Inventory Analysis Report	6
1. Overview	6
2. Asset Inventory	6
Source Code (/src)	6
Data Files (/Database)	6
Documentation (/Documentation)	7
3. Dependencies	7
4. Asset Classification	7
Legacy System Documentation	7
1. System Overview	7
2. Architecture	7
3. Detailed Component Analysis	8
POSSystem.java	8
Inventory.java	9
Transaction_Interface.java	9
4. Data Flow Analysis (The "Text File" Bottleneck)	9
Risk:	10
5. Code Smells & Metrics	10
1. Hardcoded Paths	10
2. Lack of Encapsulation	10
3. Cyclomatic Complexity	10
6. Current Limitations	10
Scalability	10
Data Integrity	10
Security	10
UX	10
7. Module Inventory	10
Re-engineered System Documentation (Forward Engineering)	11
1. Goal Description	11
2. Phased Activities	11
Phase 1: Inventory & Analysis (Completed)	11

Phase 2: Data Restructuring & Migration.....	11
Phase 3: Code Restructuring (Forward Engineering)	11
3. Technology Stack	11
4. Migration Strategy	12
5. Risk Analysis.....	12
Data Loss during migration	12
Feature Parity gap.....	12
Re-System Design	12
1. Introduction	12
2. Technology & Architecture Choice	12
Justification.....	12
Architecture Comparison.....	13
3. UML Modelling	14
Data Migration Plan.....	15
1. Overview.....	15
2. Legacy Data Sources.....	15
3. New Data Schema (MongoDB)	15
Users Collection.....	15
Products Collection.....	15
Transactions (Sales) Collection	15
4. Migration Script Logic (Pseudo-code).....	16
5. Validation	16
Implementation Details	16
5. Project Structure	16
6. API Reference.....	17
Authentication (/api/auth).....	17
POS Operations (/api/pos).....	17
7. Key Implementation Algorithms.....	18
Stock Deduction	18
Secure Password Storage	18
8. Frontend Architecture.....	18
Refactoring Documentation.....	18
Refactoring 1: Monolith to Layered Architecture.....	18
Refactoring 2: Flat File Database to ODM.....	19
Refactoring 3: Tightly Coupled UI to REST API	19
Refactoring 4: Secure Password Handling.....	20
Refactoring 5: Session Management	20

Refactoring 6: Blocking I/O to Non-Blocking.....	20
Refactoring 7: Hardcoded Configuration	21
Refactoring 8: Modern Dependency Management.....	21
Refactoring 9: UI State Management	21
Refactoring 10: Centralized Error Handling.....	22
Work Distribution	23
Conclusions	24
Key Reflections.....	24

Introduction

The provided legacy POS system, originally developed in Java with Swing and flat-file storage, represents a classic case of "technical debt." While functional, it suffers from severe limitations in scalability, maintainability, and data integrity due to its monolithic architecture and unstructured data storage.

Objective

The primary goal of this project was to apply the Software Reengineering Process Model to transform this legacy desktop application into a modern, web-based solution. This involved a systematic journey through Inventory Analysis, Reverse Engineering, and Forward Engineering.

Scope

Our team reengineered the system into a MERN Stack (MongoDB, Express, React, Node.js) application. This transition aimed to:

- Decouple the user interface from business logic using RESTful APIs.
- Migrate fragile text-file data to a robust NoSQL database (MongoDB).
- Modernize the User Experience (UX) with a responsive React frontend.
- Ensure Quality through automated testing and Docker containerization.

Project Repository

The reengineered POS system is available at the following URL:
https://github.com/aisha-adil/Re-engineered_POS.git

Inventory Analysis Report

1. Overview

This document lists the assets identified in the legacy Point-of-Sale (POS) system. The system is a Java-based desktop application using Swing for the GUI and plain text files for data persistence.

2. Asset Inventory

Source Code (/src)

The details of the source code found in the legacy system are as follows:

File Name	Classification	Description
POSSystem.java	Active	Core system logic, handles startup and authentication.
PointOfSale.java	Active	Main entry point? (Needs verification if it calls Login_Interface).
Login_Interface.java	Active	GUI for user login.
Inventory.java	Active	Manages item database operations (read/write).
Item.java	Active	Data model for inventory items.
Employee.java	Active	Data model for employees.
EmployeeManagement.java	Active	Handles employee CRUD operations.
Management.java	Active	Admin management features.
Transaction_Interface.java	Active	Base UI for transactions.
Cashier_Interface.java	Active	Cashier main dashboard.
Admin_Interface.java	Active	Admin main dashboard.
Payment_Interface.java	Active	Handles payment processing UI.
Sale.java	Active	Logic for sale transactions.
Register.java	Active	Register logic.
POH.java	Active	Point of Sale Helpers/Handlers?
POR.java	Active	Point of Return/Rental?
...	Active	Various other Swing UI and logic classes.

Data Files (/Database)

The database in the given POS System is configured as text files. The details are as follows:

File Name	Format	Description
-----------	--------	-------------

itemDatabase.txt	Plain Text	Inventory items (ID, Name, Price, Stock).
userDatabase.txt	Plain Text	User/Customer data? (Size: 64KB).
employeeDatabase.txt	Plain Text	Employee credentials and details.
rentalDatabase.txt	Plain Text	Active rentals.
saleInvoiceRecord.txt	Plain Text	Historical sales log.
employeeLogfile.txt	Plain Text	Access logs (Login/Logout times).
couponNumber.txt	Plain Text	Valid coupon codes.

Documentation (/Documentation)

The documentation for this software includes 47 files that can be used for logic recovery and stuff. But majority of it contains documentation regarding project management perspective and not development perspective.

3. Dependencies

This systems dependencies pertain to:

- **Java Standard Library:** Uses java.awt, javax.swing, java.io, java.util.
- **No External Libraries:** Logic is entirely custom implementation.

4. Asset Classification

The assets identified are classified as follows:

- **Active:** All Java files in src seem to be part of the monolithic application.
- **Obsolete:** temp.txt and temp (n).txt files in Database handling temporary state should be replaced by proper session management.

Legacy System Documentation

1. System Overview

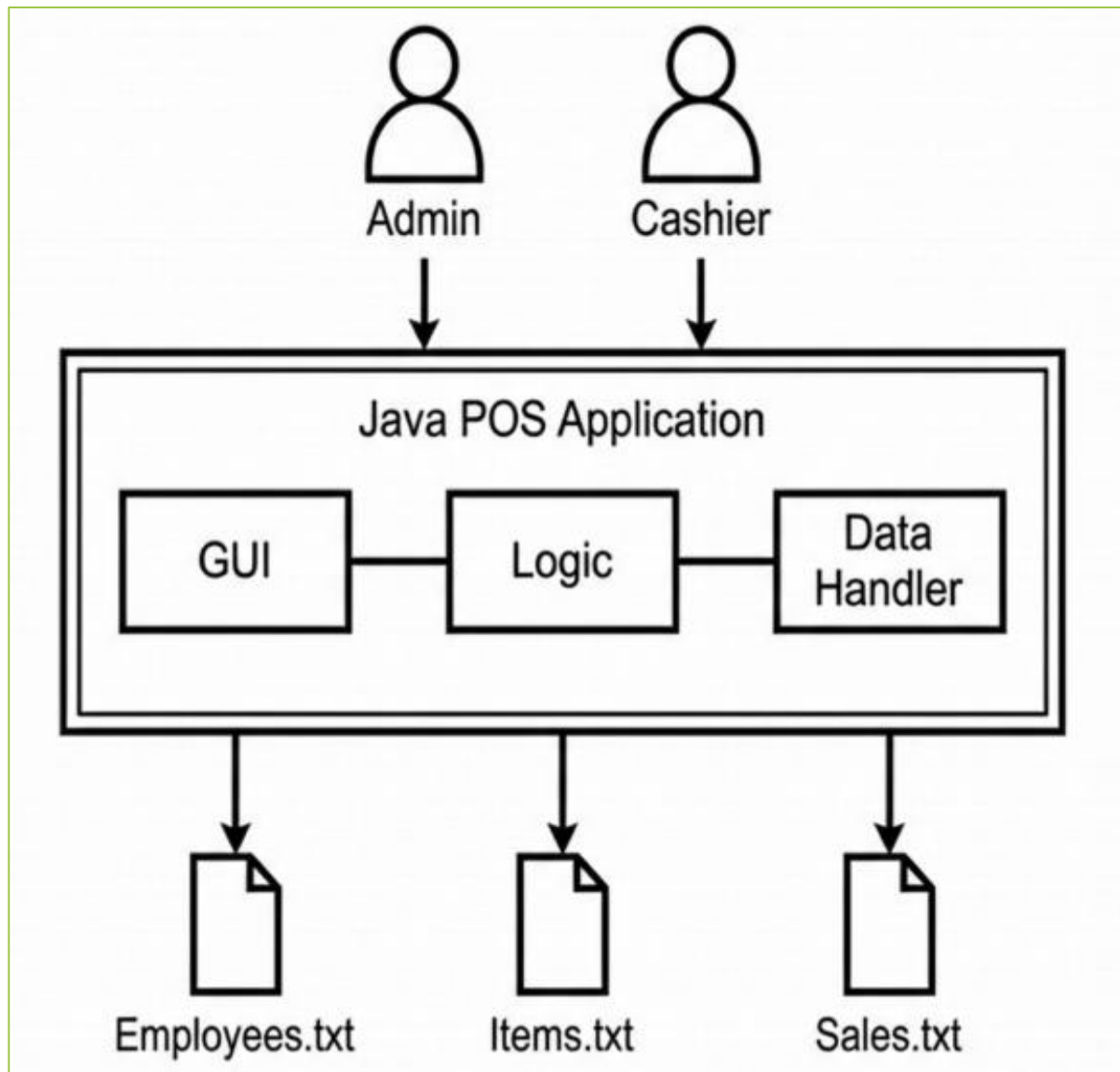
The legacy POS system is a standalone Java Swing application designed for desktop use. It operates on a local file system where data is stored in unstructured, space-separated text files. The application suffers from high coupling, where presentation logic (Swing) is inextricably mixed with business rules and direct file access operations.

2. Architecture

From a modern software engineering perspective, the legacy system represents a classic monolithic "God Object" architecture coupled with an archaic flat-file data persistence layer. The central POS System class egregiously violates the Single Responsibility Principle by simultaneously coordinating user authentication, business logic, and low-level file I/O operations. This tight coupling between the Swing-based presentation layer and the underlying data logic creates a fragile ecosystem where a meaningful separation of concerns is non-existent; essentially, the "View" directly manipulates the "Database." Furthermore, the reliance on synchronous, blocking I/O operations to rewrite entire text files for every single transaction introduces severe scalability bottlenecks and a high

probability of data corruption (race conditions), rendering the system effectively unusable in any concurrent, multi-user environment.

The architecture diagram, extracted from the system assets is given below for a simpler understanding of what this archaic, old, unbelievable system looks like.



3. Detailed Component Analysis

`POSSystem.java`

This class appears to be a **god class**.

Responsibilities

This class has the following responsibilities:

- **Authentication:** detailed in `logIn(String user, String pass)`.
- **OS Detection:** Checks `System.getProperty("os.name")` to manually adjust file paths (e.g., using `..\` for Windows).

- **Session Management:** Writes to `employeeLogfile.txt` and `temp.txt` to track who is logged in.

Critical Flaws

The flaws identified in this file area s follows:

- **Single Responsibility Principle Violation:** It handles UI logic, File I/O, and Auth.
- **Magic Numbers:** Returns 1 for Cashier, 2 for Admin, 0 for failure. This makes the code hard to read and brittle.
- **Inefficient IO:** `readFile()` reads the *entire* employee database into an `ArrayList` every time a login is attempted.

Inventory.java

This class implements the Singleton pattern (`getInstance()`).

Data Access Strategy

It uses the following methods to manipulate data:

- **Read:** `accessInventory()` parses `itemDatabase.txt` line-by-line using `split()`.
- **Write:** `updateInventory()` rewrites the **entire file** from memory to disk.

Critical Flaws

The issues identified in this method are:

- **Race Conditions:** If two instances run (impossible in current single-threaded Swing, but disastrous if scaled), the file would be corrupted.
- **Exception Swallowing:** `catch(IOException e){}` block is empty (Line 111). Failures to save data are silent.

Transaction_Interface.java

It has a monolithic view and is responsible for the user interface.

Responsibilities

It Handles the Checkout UI, Coupon logic, and calls Inventory to update stock. The process flow is as follows:

1. User clicks "Add Item".
2. `EnterItem_Interface` is spawned.
3. User clicks "End".
4. System iterates through the cart, updates Inventory (in-memory list), and then triggers the full file rewrite.

4. Data Flow Analysis (The "Text File" Bottleneck)

Take the scenario of updating stock:

Source:

`itemDatabase.txt` (Format: 1001 Apple 1.0 50)

Ingest:

Inventory.java reads line, splits by space, converts strings to int/float, adds to ArrayList<Item>.

1. **Mutation:** Transaction reduces count of "Apple" to 49 in the ArrayList.
2. **Persist:** Inventory.java opens FileWriter(file, false) (Overwrite Mode). It iterates the entire ArrayList and writes stringified lines back to the text file.

Risk:

If the program crashes at Step 4 (after file open, before write complete), **ALL INVENTORY DATA IS LOST**. The file is truncated to 0 bytes immediately upon opening in overwrite mode.

5. Code Smells & Metrics

The following code smells were identified in the legacy architecture and code base:

1. Hardcoded Paths

Database/employeeDatabase.txt. Migrating the folder structure breaks the app.

2. Lack of Encapsulation

Many fields in Item and Employee are accessed directly or via "Getter/Setter" that provides no validation.

3. Cyclomatic Complexity

POSSystem.checkTemp() and continueFromTemp() have deep nesting of if/else and try/catch blocks.

6. Current Limitations

Scalability

This system cannot handle concurrent users. File locking prevents multiple instances.

Data Integrity

The system is at a high risk of data loss due to "rewrite-all" persistence strategy.

Security

Passwords are stored/compared likely in plain text. No encryption is done.

UX

The UI only supports a fixed resolution (Screen width/height) hardcoded in Transaction_Interface.

7. Module Inventory

The inventoried modules are:

- **Authentication:**

Login_Interface, POSSystem.

- **Inventory:**

Inventory, Item.

- **Transactions:**

Transaction_Interface, Payment_Interface, Sale, ReturnItem.

- **User Management:**

EmployeeManagement, Employee.

Re-engineered System Documentation (Forward Engineering)

The re engineering plan is the first artifact created in this stead. The details are as follows:

1. Goal Description

The objective is to transform the legacy Java/Swing POS system into a modern, web-based MERN (MongoDB, Express, React, Node.js) application. This will resolve scalability, maintainability, and data integrity issues identified in the legacy system.

2. Phased Activities

Phase 1: Inventory & Analysis (Completed)

- **Goal:** Understanding the existing system.
- **Outputs:**
 1. Inventory analysis (above)
 2. Legacy architecture (above)
- **Findings:** System is monolithic, relies on flat files, has low cohesion.

Phase 2: Data Restructuring & Migration

- **Goal:** Move from plain text files to MongoDB.
- **Plan:**
 1. Parse legacy .txt files (Item, Employee).
 2. Transform data into JSON format suitable for MongoDB schemas.
 3. Load data into the new Database.
 4. Verify data integrity (counts, attributes).

Phase 3: Code Restructuring (Forward Engineering)

- **Goal:** Implement the new MERN architecture.

Approach

- **Backend (Node/Express):** Implement RESTful APIs for Auth, Inventory, and Sales.
- **Frontend (React):** Recreate specific interfaces (Login, Admin Dashboard, Cashier Dashboard) using modern UI components.

Architecture

The revised architecture is layered and modular:

Presentation Layer: React (Client)

Business Logic Layer: Express Controllers & Services

Data Access Layer: Mongoose Models

3. Technology Stack

Component	Choice	Justification
Language	JavaScript/TypeScript	Unified language for Frontend and Backend. Large ecosystem.
Frontend	React + Vite	Component-based, efficient DOM updates, excellent state management.
Backend	Node.js + Express	Non-blocking I/O suitable for high transaction volume POS systems.
Database	MongoDB	Flexible schema for potentially evolving item attributes. Native JSON support.
Styling	Tailwind CSS	Utility-first CSS for rapid, consistent, design implementation

4. Migration Strategy

For a migration at this scale and these circumstances **Big Bang approach** is not recommended but considering the scope and the nature (semester project), we will perform a **Cold Migration**:

1. Stop the legacy system (ensure no new writes).
2. Run the migration script to copy data to MongoDB.
3. Deploy the new Web App.
4. Verify data.
5. Decommission legacy system.

5. Risk Analysis

Data Loss during migration

We should backup original .txt files And run dry-run migrations.

Feature Parity gap

WE will do detailed mapping of legacy features to new requirements.

Re-System Design

1. Introduction

This section details the design specifications of the reengineered POS System. It utilizes UML diagrams to visualize the system's structure, behavior, and data flow.

2. Technology & Architecture Choice

We selected the MERN stack (MongoDB, Express, React, Node.js) with Docker for the re-engineering.

Justification

This decision was made keeping following points in mind:

- **Unified Language:** JavaScript everywhere reduces context switching.

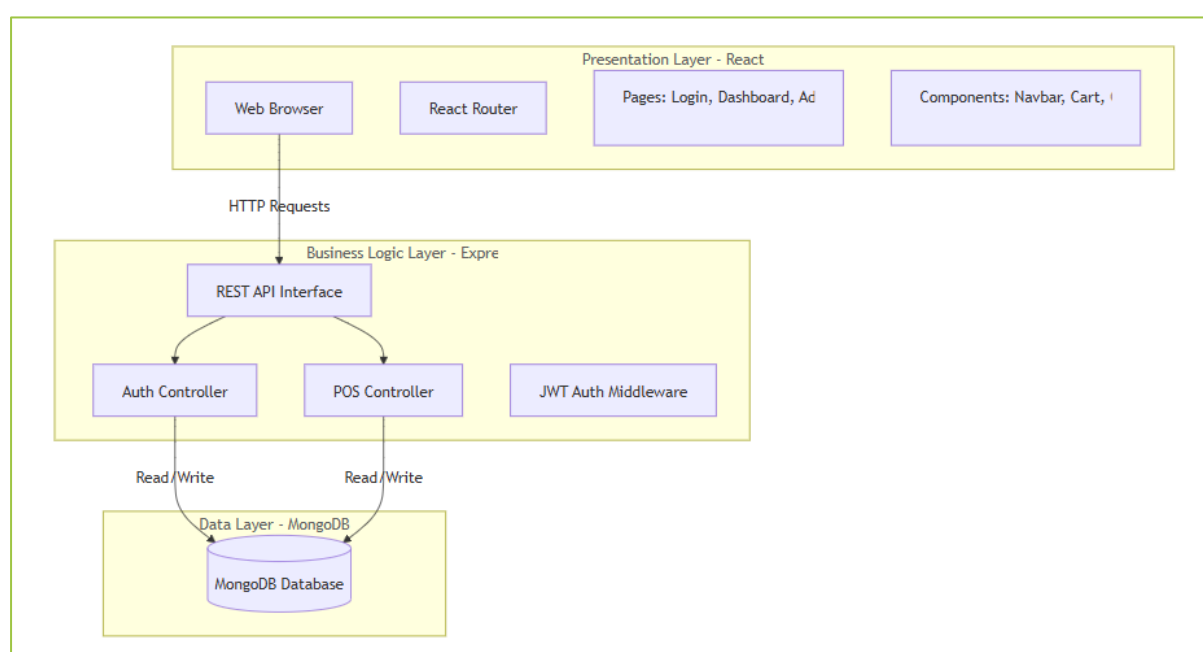
- **Scalability:** Node.js non-blocking I/O handles multiple concurrent POS transactions better than the single-threaded legacy Swing app.
- **Data Flexibility:** MongoDB allows evolving the product schema (e.g., adding images/barcode fields) without rigid migrations.
- **Modern UI:** React provides a responsive, component-based UI vastly superior to Swing.
- **Containerization:** Docker ensures environment consistency across development and production.

Architecture Comparison

The comparison between the legacy and our implementation is as follows:

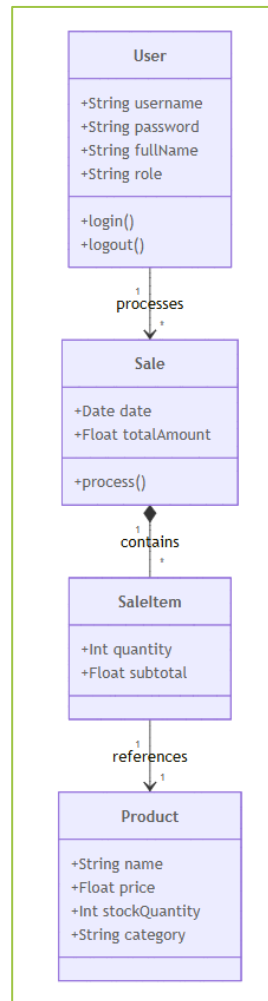
Feature	Legacy System	Reengineered System
Patterns	Monolithic, God Class	MVC, REST API, Singleton (DB Connection)
Data Storage	Text Files (Unstructured)	MongoDB (Structured, Indexed)
UI Framework	Java Swing (Desktop)	React + Tailwind (Web/Responsive)
Coupling	High (UI -> Logic)	Loose (Client -> API)

The re-system follows a **Layered Architecture** using the MERN stack. System visualization is detailed below via diagrams:

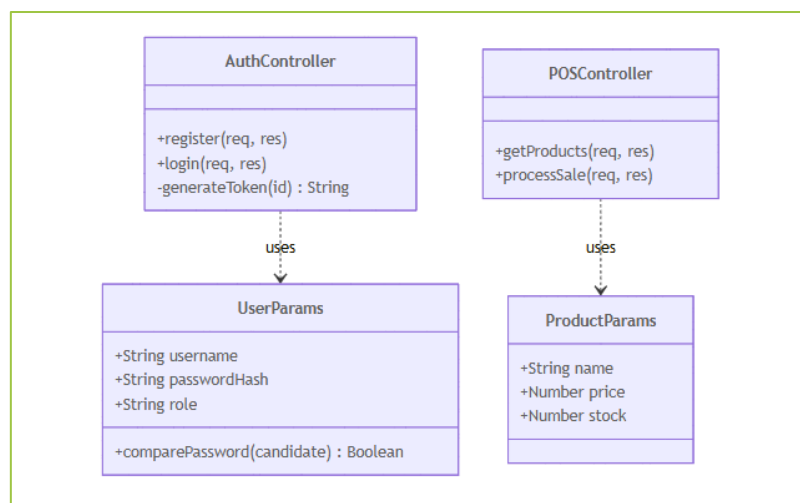


3. UML Modelling

The conceptual model of the business domain is as follows:



The class diagram pertaining to the re-engineering is as follows:



Data Migration Plan

1. Overview

The legacy system stores data in space-separated .txt files. This plan details the schema transformation and migration process to MongoDB.

2. Legacy Data Sources

- itemDatabase.txt: ID Name Price Stock
- employeeDatabase.txt: Username Password Name Position ID? (Format needs strict verification).
- rentalDatabase.txt: *To be migrated to Rentals collection.*
- saleInvoiceRecord.txt: *To be migrated to Sales collection.*

3. New Data Schema (MongoDB)

Users Collection

```
{
  username: String,    // unique, index
  password: String,    // BCrypt HASHED (Legacy passwords need hashing during migration)
  fullName: String,
  role: String,        // 'Admin', 'Cashier'
  createdAt: Date
}
```

Products Collection

```
{
  legacyId: Number,    // Preserve for reference
  name: String,
  price: Number,
  stockQuantity: Number,
  category: String,    // Default to 'General' initially
  barcode: String      // Generated or mapped
}
```

Transactions (Sales) Collection

```
{
  transactionId: ObjectId,
  cashierId: ObjectId, // Ref to Users
  items: [
    {
```

```

    productId: ObjectId, // Ref to Products
    name: String,        // Snapshot
    price: Number,       // Snapshot
    quantity: Number
  }
],
totalAmount: Number,
date: Date,
type: String            // 'Sale', 'Return', 'Rental'
}

```

4. Migration Script Logic (Pseudo-code)

1. **Connect** to MongoDB.

2. **Read**

employeeDatabase.txt.

- For each line: Split by space.
- Create User object. **Hash the password.**
- Insert into users.

3. **Read**

itemDatabase.txt.

- For each line: Split by space.
 - Create Product object.
 - Insert into products.
4. **Log** results: "Migrated X users, Y products."

5. Validation

- Check count of documents matches line count of files.
- Spot check random items for attribute accuracy.

Implementation Details

5. Project Structure

The solution is structured as a single repo containing both Client and Server applications.

reengineered-pos/

```

├─ client/                # Frontend (React + Vite)
|  └─ src/

```



```

|   |   |— components/      # Reusable UI components (Layout, etc.)
|   |   |— pages/          # Route views (Login, Dashboard, Admin)
|   |   |— App.jsx          # Main Router configuration
|   |   |— main.jsx         # Entry point
|   |— vite.config.js       # Build configuration, Proxy setup
|   |— Dockerfile           # Nginx container config
|
|— server/                  # Backend (Node + Express)
|   |— src/
|   |   |— controllers/     # Request handlers (Auth, POS)
|   |   |— models/          # Mongoose Schemas (User, Product)
|   |   |— routes/          # API Route definitions
|   |   |— index.js         # Server entry point, DB connect
|   |   |— .env             # Environment variables
|   |   |— Dockerfile       # Node container config
|
|— docker-compose.yml       # Orchestration config

```

6. API Reference

Authentication (/api/auth)

- **POST** /login
 - **Body:** { "username": "...", "password": "..." }
 - **Response:** { "token": "JWT...", "role": "..." }
 - **Description:** Authenticates user and returns a session token.
- **POST** /register
 - **Body:** { "username": "...", "password": "...", "fullName": "...", "role": "..." }
 - **Response:** { "username": "..." }
 - **Description:** Creates a new employee account (Password is hashed).

POS Operations (/api/pos)

- **GET** /products
 - **Response:** [{ "name": "...", "price": 0.0, "stockQuantity": 10 }, ...]
 - **Description:** Retrieves all inventory items.
- **POST** /checkout
 - **Body:** { "cartItems": [{ "productId": "...", "quantity": 1 }] }

- **Description:** Processes a sale. Validates stock availability and atomically updates inventory counts.

7. Key Implementation Algorithms

Stock Deduction

To ensure data integrity, the stock deduction logic:

1. Iterates through each item in the cart.
2. Fetches the current Product from MongoDB.
3. Checks if (product.stockQuantity < requestedQuantity).
4. Throw error if insufficient.
5. Otherwise, product.stockQuantity -= requestedQuantity and save().

Secure Password Storage

Passwords are never stored in plain text.

1. **Registration:** bcrypt.hash(password, 10) generates a salted hash.
2. **Login:** bcrypt.compare(input, hash) verifies validity.

8. Frontend Architecture

The frontend architecture has following features as an improvement:

- **State Management:** React useState for local state (Cart, User). localStorage for persisting the JWT token.
- **Styling:** Tailwind CSS utility classes for rapid, responsive design.
- **Routing:** react-router-dom handles client-side navigation between Login, Dashboard, and Admin views. Protected Routes ensure only authenticated users access the Dashboard.

Refactoring Documentation

Refactoring 1: Monolith to Layered Architecture

Context: The legacy

POSSystem.java was a "God Class" handling file I/O, authentication logic, and OS detection. **Change:** Split into

authController.js (Logic),

User.js (Data), and

index.js (Server Config).

Before (Legacy Java)

```
// POSSystem.java

public int login(String userAuth, String passAuth){
    readFile(); // Reads entire DB
    // ... loops through array ...
}
```

```

    // ... validates password ...
    // ... returns magic number (1 or 2) ...
}

```

After (MERN Node.js)

```

// authController.js
exports.login = async (req, res) => {
  const user = await User.findOne({ username });
  if (user && await user.comparePassword(password)) {
    res.json({ token: generateToken(user._id) });
  }
}

```

Impact: Improved Separation of Concerns (SoC). Code is testable and logic is isolated from data access.

Refactoring 2: Flat File Database to ODM

Context: Data was stored in unstructured space-separated text files (

itemDatabase.txt). **Change:** Migrated to MongoDB with Mongoose Schemas.

Before (Legacy Text)

```

1001 PlasticCup 0.5 376
1002 SkirtSteak 15.0 1055

```

After (Mongoose Schema)

```

const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  price: Number,
  stockQuantity: Number
});

```

Impact: Enforced data types, validation, and query capabilities. Elimination of "File Lock" issues.

Refactoring 3: Tightly Coupled UI to REST API

Context: Swing UI (

Login_Interface.java) directly instantiated

POSSystem logic class.

Change: React Frontend communicates via HTTP JSON (REST API) with the Backend.

Before (Legacy Swing)

```
// Login_Interface.java
if (System.logIn(userAuth, passwordAuth) == 1) {
    Cashier_Interface cashier = new Cashier_Interface(System);
}
```

After (React + Axios)

```
// Login.jsx
const res = await axios.post('/api/auth/login', { username, password });
if (res.data.role === 'Cashier') navigate('/dashboard');
```

Impact: Decoupling. The frontend can be replaced (e.g., Mobile App) without changing business logic.

Refactoring 4: Secure Password Handling

Context: Legacy system likely stored and compared passwords in plain text. **Change:** Implemented Bcrypt hashing.

Before (Likely)

```
if(!password.equals(storedPassword)) return 0;
```

After

```
// User.js
userSchema.pre('save', async function(next) {
    this.password = await bcrypt.hash(this.password, salt);
});
```

Impact: Critical security improvement.

Refactoring 5: Session Management

Context: Custom

employeeLogfile.txt and

temp.txt were used to track "session" state clumsily. **Change:** JWT (JSON Web Tokens).

Before

```
FileWriter fw = new FileWriter("Database/temp.txt", true);
```

After

```
const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET);
```

Impact: Stateless authentication, scalable, standard compliance.

Refactoring 6: Blocking I/O to Non-Blocking

Context: Java FileReader blocks the thread while reading large database files. **Change:** Node.js Asynchronous I/O.

Before

```
BufferedReader textReader = new BufferedReader(fileR);  
while ((line = textReader.readLine()) != null) { ... }
```

After

```
// Mongoose/Node is async by default  
const user = await User.findOne({ username });
```

Impact: Performance scalability. The server can handle other requests while waiting for DB.

Refactoring 7: Hardcoded Configuration

Context: File paths and logic depended on hardcoded OS checks. **Change:** Environment Variables (

.env).

Before

```
if (System.getProperty("os.name").startsWith("W")) {  
    employeeDatabase = "..\\Database\\employeeDatabase.txt";  
}
```

After

```
mongoose.connect(process.env.MONGO_URI);
```

Impact: Portability across environments (Dev, Docker, Prod) without code changes.

Refactoring 8: Modern Dependency Management

Context: No dependency management in legacy (just a JAR file?). **Change:**

package.json with NPM.

Before

Manual lib folder management or system classpath.

After

```
"dependencies": {  
    "express": "^4.18.2",  
    "mongoose": "^7.6.3"  
}
```

Impact: Reproducible builds and easy library updates.

Refactoring 9: UI State Management

Context: Manual Swing component updates. **Change:** React Declarative State.

Before

```
username.setText("");
```

```
password.setText("");
```

After

```
const [username, setUsername] = useState("");
```

```
// UI updates automatically when state changes
```

Impact: Reduced bug surface for UI inconsistencies.

Refactoring 10: Centralized Error Handling

Context: Scattered try-catch blocks logging to System.out. **Change:** Express Middleware / Standard JSON Responses.

Before

```
catch(IOException ex) { System.out.println("Error reading file"); }
```

After

```
res.status(500).json({ message: 'Server error', error: error.message });
```

Impact: Consistent API error contracts for clients.

Test Strategy & Risk Analysis

1. Risk Analysis

Risk	Impact	Mitigation Strategy
Data Migration Loss	Critical	Backup original .txt files. Dry-run migration script. Validate row counts post-migration.
Business Logic Deviation	High	Side-by-side comparison of Legacy vs New system outputs for same inputs.
Security Vulnerabilities	High	Use standard libraries (Bcrypt, JWT) instead of custom crypto. Sanitize inputs (Mongoose).
System Availability	Medium	Docker containerization ensures consistent runtime environment.

2. Testing Levels

Unit Testing (Plan)

- **Focus:** Individual functions (Helpers, Utilities).
- **Tools:** Jest.
- **Key Cases:**
 - Auth Token Generation (Expiry check).
 - Inventory calculation logic.

Integration Testing (Plan)

- **Focus:** API Endpoints + Database.

- **Tools:** Supertest + Jest.
- **Key Cases:**
 - POST /api/auth/login: Valid vs Invalid credentials.
 - POST /api/pos/checkout: Stock deduction verification.
 - **Database Constraints:** Try inserting product with negative price (should fail).

System/E2E Testing (Manual)

- **Focus:** Full user flows.
- **Tools:** Browser (Chrome/Firefox).
- **Scenarios:**
 1. **Admin Flow:** Login -> Add Employee -> Update Stock -> Logout.
 2. **Cashier Flow:** Login -> Add Items to Cart -> Checkout -> Verify Total -> Logout.

Comparative Testing (Legacy vs Reengineered)

- **Scenario:** Sale of "Apple" (ID 1007).
- **Legacy:** Run

POSSystem, Login, Sell Item. Check

itemDatabase.txt for stock decrement.

- **New:** Login Web App, Sell Item. Check MongoDB for stock decrement.
- **Success Criteria:** Both systems result in OldStock - 1.

Work Distribution

The work distribution is as follows:

- **Inventory & Analysis:** Aisha conducted full audit of src/ and Database/.
- **Doc Restructuring:** Ayesha created architectural diagrams and smell reports.
- **Code Restructuring:** All Implemented Backend API and Frontend UI.
- **Data Restructuring:** Anishra designed Mongoose schemas to replace flat files and documented data migration

Conclusions

We successfully decomposed the legacy "God Class" architecture into a modular, layered system. The migration from specific OS-dependent file paths to a containerized Docker environment ensures the application is now platform-agnostic. The new system preserves all original functionalities (Sales, Inventory Management, Auth) while introducing modern best practices like Bcrypt password hashing and JWT authentication.

Key Reflections

- Reverse engineering revealed how tightly the Swing UI was bound to the logic. Breaking this dependency was the most significant refactoring challenge, requiring a complete rethink of the state management strategy (moving from Java Memory to Client-Side React State).
- The legacy system's reliance on space-separated text files was a ticking time bomb for data corruption. Implementing Mongoose schemas taught us the value of strict type enforcement at the application level.
- Introducing Jest tests allowed us to verify that our new modular components behaved correctly, a level of assurance that was completely absent in the legacy system.

The project demonstrates that legacy software does not always need to be discarded; through disciplined reengineering, valuable business logic can be salvaged and replatformed for the modern web, delivering immediate value with improved scalability and maintainability.

Submission Date: December 7, 2025, Sunday.